

Software Requirements Specification

LibraQuiz Enterprise Academic Platform

Version 2.0 Approved (Microservices Architecture)

Prepared By	Vinit Lokhande, Bhushan Khandekar, Kanchan Dhete, Piyush Raut
Organization	CDAC Authorized Training Centre Soft Polynomials Pvt. Ltd., Nagpur
Date Created	August 9, 2026

Revision History

Name	Date	Reason For Changes	Version
Dev Team	2026-06-01	Initial draft merging Project P1 (Library) and Project P3 (Quiz Engine).	0.1
Arch Committee	2026-06-15	Upgraded tech stack to modern React and Spring Boot architecture.	0.5
Project Lead	2026-07-09	Final review and base authorization approval.	1.0 Approved
Arch & Dev Team	2026-08-09	Upgraded architecture from monolithic Spring Boot to Spring Cloud 14-Microservices ecosystem with Eureka Server, API Gateway, AIService, multi-database MySQL 8.0 schema, and Redux Toolkit React SPA.	2.0 Approved

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the comprehensive functional and non-functional requirements for the LibraQuiz Enterprise Academic Platform. This document establishes the formal specification for software release version 2.0, consolidating previous standalone designs into a modern, 14-microservice Spring Cloud backend ecosystem integrated with a React 18 single-page frontend application.

1.2 Document Conventions

- Standard software engineering documentation metrics govern this document.
- Heading hierarchies strictly dictate the priority structural flow.
- High-level requirements explicitly inherit priority criteria down to detail-level implementations.
- Text formatted in **bold** highlights critical technical objects, system modules, or explicit database constraints.

1.3 Intended Audience & Reading Suggestions

- Target Audience:** Project developers, academic evaluators, database architects, and system administrative staff.
- Reading Suggestion:** Developers and system architects should start with Section 2 (Overall Description) and Section 3 (External Interfaces) before analyzing the granular system features outlined via explicit functional specifications in Section 5.

1.4 Product Scope

LibraQuiz bridges the functional gap between learning resource consumption and knowledge evaluation. The system enables students to explore physical/digital library reference assets, enroll in courses, and immediately execute targeted online examinations created by faculty members. Version 2.0 introduces AI-assisted search capabilities (AIService), Fisher-Yates randomized quiz creation, automated retake attempt limits, and strict teacher/student data isolation.

1.5 References

- **1. Project P1 Specifications** – Library Records System (Bhushan + Vinit), Soft Polynomials Pvt. Ltd.
- **2. Project P3 Specifications** – Quiz / Examination Engine (Kanchan + Piyush), Soft Polynomials Pvt. Ltd.
- **3. Karl E. Wieggers** – SRS Template Architecture Standards (1999).

2. Overall Description

2.1 Product Perspective

LibraQuiz Version 2.0 represents a modern, microservices-based distributed platform. Built on Spring Cloud and React 18, it features Service Discovery (Eureka Server), API Gateway routing, isolated MySQL databases per service domain, and stateless JWT token authentication. The architecture guarantees high scalability, fault isolation, and low latency for concurrent student examination sessions.

2.2 Product Functions

- **Authentication & Authorization:** Centralized multi-role access control (STUDENT, TEACHER, LIBRARIAN, ADMIN) managed centrally by IdentityService with Spring Security and JWT token generation.
- **Book & Inventory Management:** Physical and digital catalog management, barcode copy tracking (BC-...), ISBN classification, and physical inventory controls via BookService.
- **Member Management:** Lifecycle tracking for Students, Faculty, Librarians, and Administrators.
- **Issue & Return System:** LibraryTransactionService handles checkout leases, due date tracking, automated overdue fine calculation, and return processing.
- **Question Bank & Quiz Studio:** QuestionBankService seeded with 100 MCQs across 5 core topics (Java, .NET, DSA, Database, OS) with Easy/Medium/Hard difficulty filters, custom faculty question appends, and Fisher-Yates random sampling.
- **Automated Examination Engine:** QuizService and ExaminationService enforce live countdown timers, 4-option MCQ selection cards, automated scoring, passing thresholds (50%-80%), and configurable retake attempt limits (1-3 attempts).
- **Student Analytics & Deterministic Retakes:** ResultService tracks score histories scoped per student (`lq_student_results_${studentKey}`) and enforces deterministic question locking (`lq_quiz_questions_${quizId}`) during retakes.

2.3 User Classes and Characteristics

User Class	Access Privilege Level	Core Responsibility & Behavior
Admin	Full System Access	System-wide administrative console, user onboarding, role mapping (STUDENT, TEACHER, LIBRARIAN, ADMIN), category management, and system metrics.
Librarian	Library Operations Access	Physical/digital library inventory desk, barcode/copy code management (BC-...), issue/return processing, overdue tracking, and fine management.

User Class	Access Privilege Level	Core Responsibility & Behavior
Teacher / Faculty	Educational Management	Question bank studio (100+ MCQ dataset + custom question builder), Quiz Auto-Builder with Fisher-Yates sampling, attempt limit rules, and teacher isolation (createdByTeacher).
Student	End-User Transactional	Book discovery, borrowing requests, course enrollment, live timed quiz execution, deterministic retake engine, score breakdown, and AI-powered search.

2.4 Operating Environment

- **Backend Microservices Stack:** JDK 17, Spring Boot 3.2.3, Spring Cloud 2023.0.0 (Eureka Server on port 8761, API Gateway on port 8080).
- **Database Tier:** MySQL 8.0 relational server hosting 4 isolated databases (lq_identitydb, lq_bookdb, lq_questionbankdb, lq_quizdb).
- **Frontend SPA Client:** React 18, Vite (port 5173), Redux Toolkit (authSlice), Axios HTTP client with JWT interceptors, Lucide icons, and Glassmorphism CSS.

2.5 Design and Implementation Constraints

- Strict adherence to a Spring Cloud microservices paradigm.
- Communication between the React SPA and backend services is routed through ApiGateway using RESTful JSON endpoints secured by JWT Bearer tokens.
- Data randomization uses the Fisher-Yates shuffle algorithm in QuizService.
- Exam retake fairness is guaranteed via immutable locked question snapshots stored under lq_quiz_questions_{quizId}.

2.6 User Documentation

- Inline interactive markdown manual.
- Embedded system deployment checklist for administrators.
- Dynamic modal help tips within the Librarian and Student dashboard interfaces.

2.7 Assumptions and Dependencies

- Institutional client systems maintain steady TCP/IP network access to the hosted MySQL server.
- Scoring, timing, and notification mechanisms depend heavily on consistent, synchronized system clocks across runtime application server threads.

3. External Interface Requirements

3.1 User Interfaces

The presentation layer utilizes a responsive, clean, institutional web layout built on React. Navigational headers provide clear status readouts (e.g., active borrow count, pending fines, or active quiz countdown timer). All lists, ledger data tables, and leaderboards employ server-side pagination with explicit data filtering. Forms include client-side validation thresholds preventing malformed submissions prior to API dispatching.

3.2 Hardware Interfaces

The platform requires no proprietary hardware interfaces, interacting natively via standard TCP/IP networking sockets over standard client-server ports.

3.3 Software Interfaces

- **Database System:** MySQL 8.0 hosting isolated microservice databases (lq_identitydb, lq_bookdb, lq_questionbankdb, lq_quizdb) managed via Spring Data JPA / Hibernate.
- **Security & Middleware Layer:** EurekaServer for service discovery, ApiGateway for request routing, and Spring Security for JWT stateless token validation.
- **Frontend & Build Stack:** React 18, Redux Toolkit, Vite build tool, Node.js/NPM package ecosystem, Apache Maven multi-module parent pom.xml, and Git repository control.

3.4 Communications Interfaces

- All communication channels across endpoints occur over secure HTTP protocols (HTTPS).
- API responses use standard content-type parameters transmitting structured JSON payloads.
- Automated asynchronous notification threads connect to institutional SMTP servers to dispatch email alerts regarding fine thresholds or quiz schedules.

4. Domain Model

Table Name	Core Fields & Constraints	Relational Association Mapping
lq_identitydb (Users / Roles)	user_id (PK), username, email, password_hash, role (STUDENT, TEACHER, LIBRARIAN, ADMIN)	Central IdentityService microservice. Issues JWT tokens and manages multi-role authentication headers.
lq_bookdb (Books & Transactions)	book_id (PK), title, isbn, author, total_copies, copy_code (BC-...), issue_id (PK), user_id (FK), issue_date, due_date, return_date, fine_amount	BookService & LibraryTransactionService. Tracks physical copy inventory codes and student borrow lease ledgers.
lq_questionbankdb (Question Repository)	question_id (PK), topic (Java, .NET, DSA, Database, OS), difficulty (Easy, Medium, Hard), question_text, option_a, option_b, option_c, option_d, correct_option	QuestionBankService. Seeded with 100 high-quality computer science MCQs via SeedDataConfig.java plus teacher appends.
lq_quizdb (Quiz & Examination)	quiz_id (PK), title, topic, duration_minutes, pass_score, max_attempts (1-3), is_published, created_by_teacher, attempt_id (PK), student_id (FK), score, attempt_count	QuizService, ExaminationService & ResultService. Manages quiz creation, locked question snapshots (lq_quiz_questions_\${quizId}), and retake tracking.
lq_coursedb (Courses & Categories)	course_id (PK), category_id (PK), course_name, description, enrolled_students	CourseService & CategoryService. Taxonomy organization and student academic enrollment tracking.

5. System Features (Use Cases)

Use Case 1: Issue Book

Goal: Authorize and record the borrowing of a cataloged book asset by an eligible student or faculty member.

Input: User ID, Book ID.

Output: Confirmed transactional lease receipt, updated inventory index, and generated return target parameters.

Pre-condition: The user must maintain a status clear of outstanding fines. The requested book asset must possess an available copy index greater than zero.

Main Scenario:

- 1. The Librarian enters the borrower's identifier profile within the UI panel.
- 2. The system evaluates outstanding fines and active checkout balances against the user's specific limits.
- 3. The Librarian scans or selects the targeted Book ID identifier.
- 4. The system decrements the available copy count and appends an active lease ledger record to the Issued_Books table.

Exceptional Scenario:

If the user holds overdue materials or active fines, the transaction halts immediately and dispatches an alert constraint blocking completion until the fine is cleared.

Use Case 2: Attempt Timed Quiz

Goal: Provide students a secure environment to execute evaluations and register score metrics.

Input: Quiz Selection ID, Sequential Answer Input Options.

Output: Persisted scoring metrics, real-time leaderboard update, and an auto-generated HTML breakdown log.

Pre-condition: Student authenticated status verified. The target quiz must contain valid active questions.

Main Scenario:

- 1. The Student selects an available quiz category.
- 2. The system extracts questions from the database and randomizes selection order via a backend implementation of the Fisher-Yates shuffle algorithm.
- 3. A terminal countdown clock initializes on the frontend interface based on the defined quiz parameters.
- 4. Questions are rendered sequentially; options are logged sequentially as chosen.
- 5. Upon manual completion or clock expiration, the response object is pushed to the backend scoring logic.
- 6. The system computes scores, applies a high-performance Merge/Quick sort mechanism to update historical user rows, and re-renders the leaderboard views.

Use Case 3: Faculty Quiz Auto-Builder Studio

Goal: Enable teachers and faculty members to create, customize, sample, and publish targeted online quizzes with automated question selection and attempt rules.

Input: Quiz Title, Subject Topic (Java, .NET, DSA, Database, OS), Duration (15-60 mins), Question Limit (5-20), Difficulty (Easy/Medium/Hard/All), Pass Score % (50%-80%), and Retake Attempt Limit (1-3 attempts).

Output: Persisted Quiz entity in `lq_quizdb`, locked question snapshot under `lq_quiz_questions_{quizId}`, and published state badge visible in Student Hub.

Pre-condition: Faculty user authenticated with TEACHER role.

Main Scenario:

- 1. The Teacher opens the Quiz Builder Studio modal from the Teacher Dashboard.
- 2. The Teacher specifies quiz parameters including topic, question count, duration, pass threshold, and max retakes (1-3).
- 3. The backend QuizService executes the Fisher-Yates random sampling algorithm against `lq_questionbankdb` to select matching questions.
- 4. The generated quiz initializes in DRAFT/UNPUBLISHED status scoped strictly under `createdByTeacher: ${teacherKey}`.
- 5. The Teacher reviews the sampled questions and clicks Publish Quiz, making it accessible to enrolled students.

Exceptional Scenario:

If insufficient questions match the chosen topic and difficulty criteria, the engine prompts an alert requiring the Teacher to adjust filters or append new questions to the bank.

Use Case 4: AI-Powered Knowledge Search & Recommendation Engine

Goal: Provide students and faculty with intelligent AI search capabilities for digital catalog discovery, topic summaries, and smart study recommendations.

Input: Natural language query prompt, target subject filters.

Output: Contextually relevant book suggestions, subject overview summaries, and recommended practice quizzes.

Pre-condition: User logged in; AIService microservice online.

Main Scenario:

- 1. The Student or Teacher navigates to the AI Search page.
- 2. The user enters a natural language search query (e.g., 'Find core Data Structures books and sample practice quizzes').
- 3. The API Gateway routes the query payload to the AIService microservice.
- 4. AIService analyzes query intent, queries `lq_bookdb` and `lq_questionbankdb`, and generates structured recommendations.
- 5. Results are rendered on the UI with direct action links to borrow books or start matching quizzes.

Exceptional Scenario:

If the AIService is temporarily unreachable, the frontend gracefully falls back to standard indexed keyword search.

6. Other Nonfunctional Requirements

6.1 Performance Requirements

The system must process response routing timelines under 200ms for standard database lookups. Under intense concurrent evaluation stress (e.g., synchronous testing windows), the platform must guarantee smooth data collection execution without dropping active socket links.

6.2 Safety Requirements

No operational process should cause data loss or table truncation during a sudden infrastructure failure. Database updates must use isolated, rolling ACID transactional processes to ensure absolute database integrity.

6.3 Security Requirements

All sensitive information passwords must undergo cryptographic one-way hashing via high-strength algorithms (e.g., BCrypt) prior to persistent storage. Access tokens must carry strict timing signatures to invalidate inactive client connections automatically.

6.4 Software Quality Attributes

The unified architecture emphasizes strict Robustness and Maintainability. The backend Spring Boot configuration must isolate transactional database tasks cleanly from core analytical computing tasks, allowing modular code adjustments without breaking upstream system logic.

6.5 Other Requirements

The system layout must fully support localized standard formatting rules regarding timezone calculations, international currency syntax for penalty ledgers, and standard numerical presentation structures across administrative metric graphs.

Appendices

Appendix A: Glossary

- **PK / FK:** Primary Key / Foreign Key indices governing relational table structures.
- **JWT:** JSON Web Token utilized for secure, stateless client authorization handshakes.
- **Fisher-Yates:** A linear complexity algorithm utilized to generate completely unbiased random permutations of a data array sequence.

Appendix B: Analysis Models

Entity relationships comply with normalized 3NF standards to preserve transactional scaling and guarantee index acceleration during high-volume query joins across user logs, catalog trackers, and scoring tables.

Appendix C: Fully Implemented v2.0 Features

- **AI Knowledge Search & Assistance:** AIService provides intelligent catalog querying, automated recommendation generation, and smart search assistance.
- **AI Search & Assistance Engine:** Fully implemented in Version 2.0 via the AIService microservice and frontend AISearchPage client.